SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

CareFlow: AI-Powered Patient Flow Optimizer

Team we hack UM | umhackathon@um.edu.my

April 2026

1. Introduction

1.1. Purpose

This System Analysis Document (SAD) details the technical scope and architectural decisions behind CareFlow — an AI-Powered Patient Flow Optimizer built for UMHackathon 2026. CareFlow is designed to address inefficiencies in hospital patient intake and triage workflows by integrating a large language model (LLM) directly into the clinical pathway.

The key elements covered in this document are:

- Architecture: A cloud-native, service-oriented platform deployed on cloud infrastructure, with a clear separation between the patient-facing intake layer, the AI triage engine, and the clinician dashboard.
- Data Flows: How patient-submitted symptom data travels from the intake form, is processed by the LLM triage module, and results in a structured triage recommendation stored in the database and surfaced to clinical staff.
- Model Process: End-to-end workflow covering patient intake → AI-powered triage → smart ward/doctor routing → insurance pre-authorisation → clinician copilot review.
- Role of Reference: This document serves as the primary technical reference for all developers, QA personnel, and evaluators at UMHackathon 2026.

1.2. Background

Malaysian public hospitals face severe patient flow bottlenecks, especially at Emergency Departments (EDs) and outpatient registration counters. Manual triage and paper-based intake lead to misrouting, long wait times, and preventable clinical errors. The multilingual patient population (Bahasa Malaysia & English) further complicates intake accuracy when relying on non-standardised verbal assessments.

CareFlow was conceived to solve these problems holistically:

- Previous State: Registration and triage were entirely manual — paper forms, non-standardised nurse assessments, and verbal handoffs between departments.
- Major Architectural Change: Introduction of an AI triage layer powered by an LLM (Groq and OpenAI API Key) that parses bilingual symptom descriptions and outputs a structured triage level (P1–P5) with a routing recommendation.
- New Capabilities: Automated insurance pre-authorisation checks, real-time bed/doctor availability routing, and an AI clinician copilot that summarises patient records for attending physicians.

1.3. Target Stakeholders

Stakeholder	Role	Expectations
Patient	Submits symptoms and demographic info via the multilingual intake portal (BM / English)	Fast, clear triage acknowledgement; reduced wait time; minimal data re-entry
Nurse / Triage Staff	Reviews AI-generated triage suggestions and confirms or overrides them	Accurate AI triage output; one-click override; real-time queue visibility
Attending Physician (Clinician)	Uses the AI copilot to review patient summaries and proposed diagnoses before consultation	Concise, structured patient summaries; source-cited reasoning from LLM
Insurance Coordinator	Reviews automated pre-authorisation requests generated by the system	Accurate procedure codes; correct patient insurance details; audit trail
Hospital Admin	Monitors patient flow dashboards and resource utilisation	Real-time occupancy stats; routing efficiency metrics; SLA compliance
Development Team	Builds and maintains the platform	Clear API contracts; modular codebase; documented LLM prompt schemas
QA Team	Validates system behaviour across all user roles	Defined request/response formats; edge case coverage; triage accuracy benchmarks

2. System Architecture & Design

2.1. High-Level Architecture

2.1.1. Overview

Type	Details
System	Web Application (React / Next.js) + REST API Backend
Architecture	Cloud-hosted monolith with service-oriented LLM integration layer
LLM Provider	OpenAI (ilmu-glm-5.1 model)+ Groq (llama-3.3-70b model)
Database	PostgreSQL (Neon / Supabase cloud)

CareFlow is structured as a client-server system with three primary client surfaces: (1) the Patient Intake Portal (web, bilingual BM/EN), (2) the Nurse/Triage Dashboard, and (3) the Clinician Copilot Interface. All clients communicate with a unified backend REST API. The backend routes requests through a dedicated LLM Service layer before persisting results to the PostgreSQL database.

2.1.2. LLM as Service Layer

The LLM (Groq and OpenAI API) is not a generic 'AI module' — it is a structured service layer with three distinct call types:

- Triage Inference Call: Patient symptom text (BM or EN) + patient vitals are injected into a structured system prompt. Groq and OpenAI returns a JSON object: { triageLevel: 'P2', urgencyReason: '...', recommendedDepartment: '...', suggestedActions: [...] }
- Insurance Pre-Auth Call: Patient demographics + ICD-10 code (proposed) + insurance policy ID are sent. Groq and OpenAI returns a structured pre-auth request body formatted for the insurer API.
- Clinician Copilot Call: Full patient history (past visits, medications, allergies) + current intake data are sent with a summarisation prompt. Groq and OpenAI returns a concise clinical note.

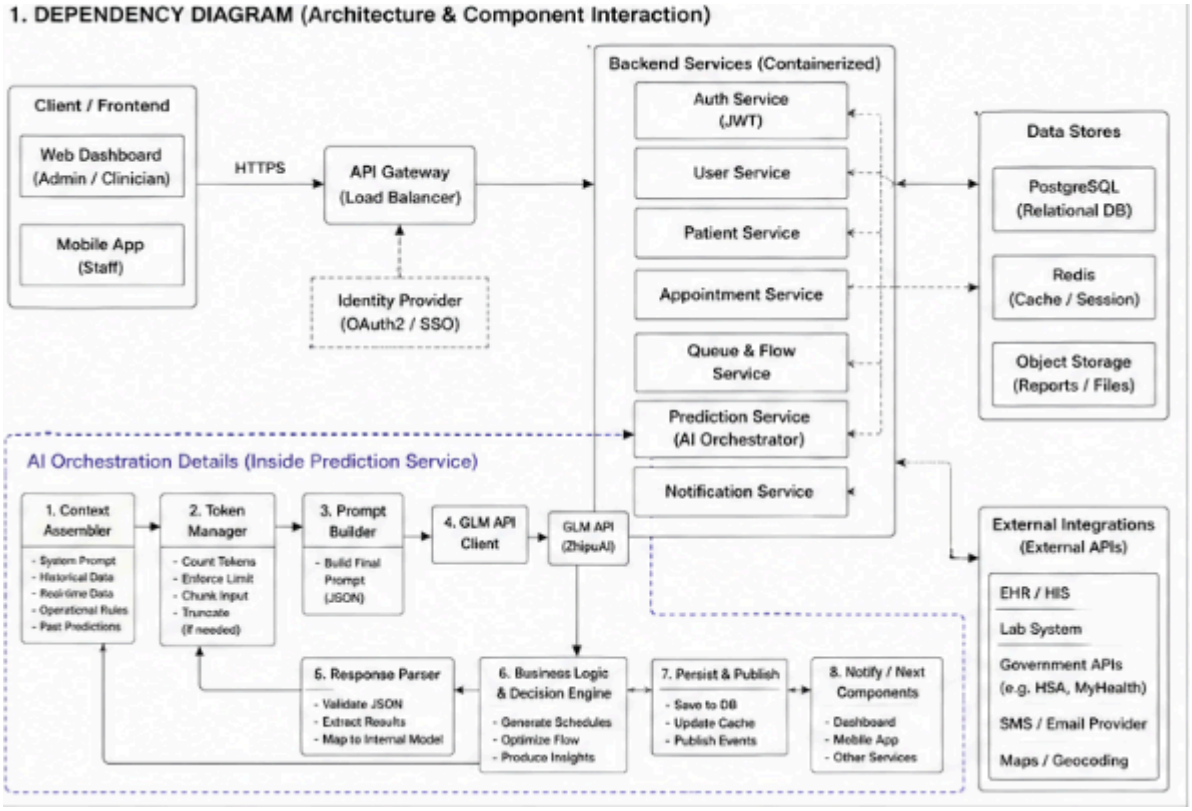
Context Window Management: Patient history older than 3 visits is summarised and compressed before injection. Raw intake text is truncated to 500 tokens. Repeated triage queries are cached (Redis, TTL 30 min) to reduce API costs.

2.1.3. Dependency Diagram

The following describes the major inter-component API calls and data flows:

From	To	Call Type	Purpose
Patient Intake Portal (Next.js)	Backend API (Node.js / Express)	HTTPS POST /api/intake	Submit patient symptoms + demographics
Backend API	LLM Service Module	Internal function call	Build and dispatch triage prompt
LLM Service Module	Anthropic Groq and OpenAI API	HTTPS POST /v1/messages	Execute triage / copilot inference
Anthropic Groq and OpenAI API	LLM Service Module	JSON Response (SSE stream)	Return structured triage JSON
LLM Service Module	Backend API	Parsed JSON object	Return triage result to router
Backend API	PostgreSQL (Neon)	SQL INSERT / SELECT	Persist patient record, triage result
Backend API	Nurse Dashboard (Next.js)	HTTPS GET /api/queue	Push triage queue and patient list
Clinician Dashboard	Backend API	HTTPS GET /api/patient/:id/summary	Request AI copilot summary

Backend API	Insurance API (External)	HTTPS POST /preauth	Submit pre-auth request
Backend API	Redis Cache	GET / SET	Cache frequent triage results



2.1.4. Prompt Construction Details

a. System Prompt: Contains role definition ('You are a clinical triage assistant...'), output schema (strict JSON), language instruction (respond in English regardless of input language), and safety guardrails (never diagnose, always recommend physician review).

b. User Message: Structured as: { language: 'BM' | 'EN', symptoms: '...', vitals: { BP: '...', HR: '...', SpO2: '...' }, age: 34, gender: 'M' }

c. Context Window: Max 4,096 tokens per call. Patient history is pre-summarised by a lightweight summary prompt if it exceeds 1,500 tokens before injecting into the triage call.

d. Token Limit Enforcement: Input is chunked at the LLM Service layer before reaching the . Soft limit = 3,500 tokens (leaving room for OpenAI (ilmu-glm-5.1 model)+ Groq (llama-3.3-70b model)); hard timeout = 8 seconds.

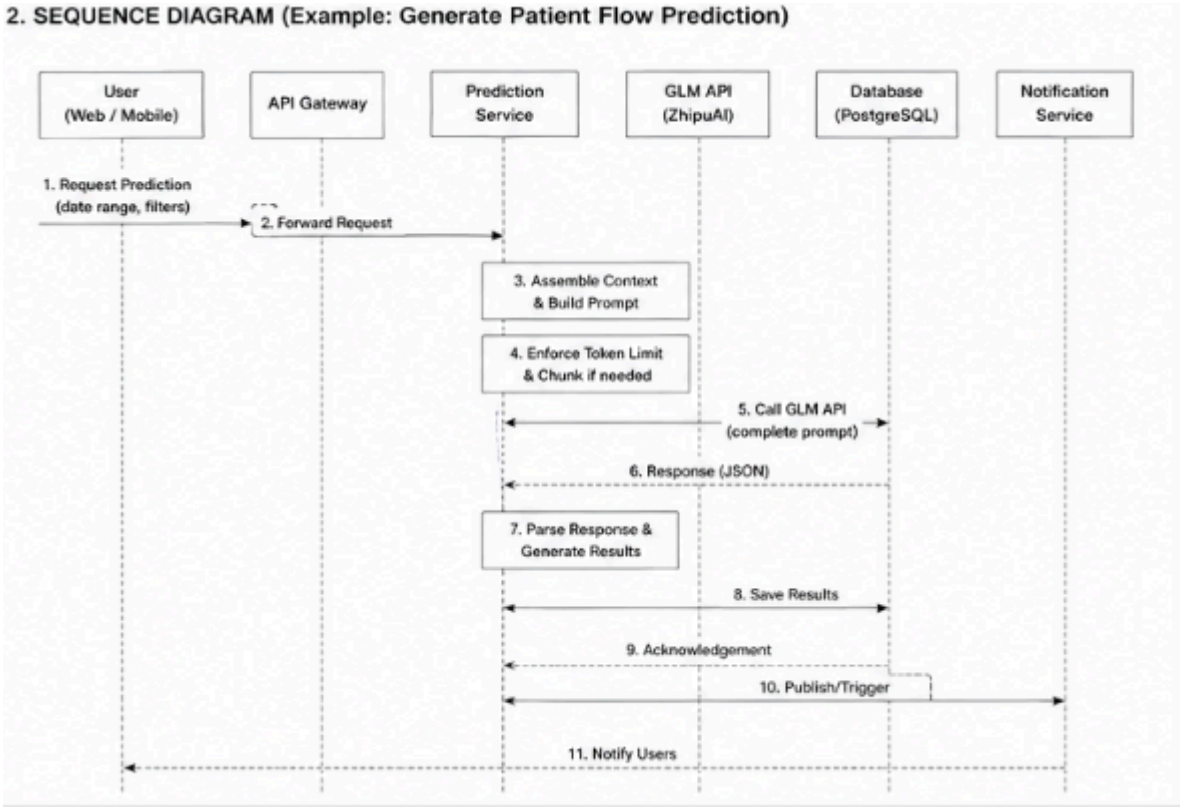
e. Response Parsing: Groq and OpenAI is instructed to return only valid JSON. The LLM Service wraps the call in a try/catch and falls back to a 'degraded mode' response (standard P3 triage) if parsing fails.

2.2.2. Sequence Diagram — Patient Intake & AI Triage Flow

The following describes the step-by-step interaction for the primary use case (Patient submits intake → AI triage → Nurse review):

Step	Actor	Action / System Response
1	Patient	Opens Patient Intake Portal; selects language (BM / EN)
2	Patient	Fills multilingual intake form: name, IC, symptoms, duration, vitals (if self-measured)
3	Intake Portal	Validates form fields client-side; sends HTTPS POST /api/intake to Backend API
4	Backend API	Validates payload; creates a pending PatientRecord in PostgreSQL (status: 'pending_triage')
5	LLM Service	Constructs triage prompt: system prompt + structured patient JSON; enforces 3,500 token limit
6	OpenAI and Groq API	Receives prompt; streams JSON response with triageLevel, urgencyReason, recommendedDepartment
7	LLM Service	Parses Groq and OpenAI response; validates JSON schema; falls back to P3 if malformed
8	Backend API	Updates PatientRecord: stores triage result (status: 'triaged'); triggers queue update event
9	Nurse Dashboard	Receives WebSocket push (or polling refresh); displays patient in triage queue with AI recommendation

10	Nurse	Reviews AI triage suggestion; confirms or overrides triage level with one-click action
11	Backend API	Records final triage decision (human-confirmed); routes patient to correct department/bed
12	Patient	Receives SMS/screen notification with queue number and estimated wait time



2.2. Technological Stack

Layer	Technology	Justification
Frontend	Next.js 14 (React)	SSR for fast initial load; built-in API routes; Vercel deployment synergy; TypeScript support

UI Styling	Tailwind CSS + shadcn/ui	Rapid prototyping; accessible component library; consistent design tokens
Backend	Node.js + Express.js	JavaScript end-to-end with frontend; fast REST API; wide ecosystem for LLM SDK integration
LLM Integration	Groq and OpenAI API	Official API key; supports streaming; structured JSON output mode
Database	PostgreSQL (Neon)	Relational schema for structured patient records; serverless-friendly via Neon; ACID compliance
ORM	Prisma	Type-safe DB queries; schema migrations; integrates cleanly with TypeScript
Caching	Redis (Upstash)	Serverless Redis; caches frequent triage patterns; reduces redundant LLM API calls
Authentication	NextAuth.js / JWT	Session management for nurses and clinicians; role-based access control
Language Detection	langdetect (JS port)	Detects BM vs EN input before prompt construction; enables language routing

2.3. Key Data Flows

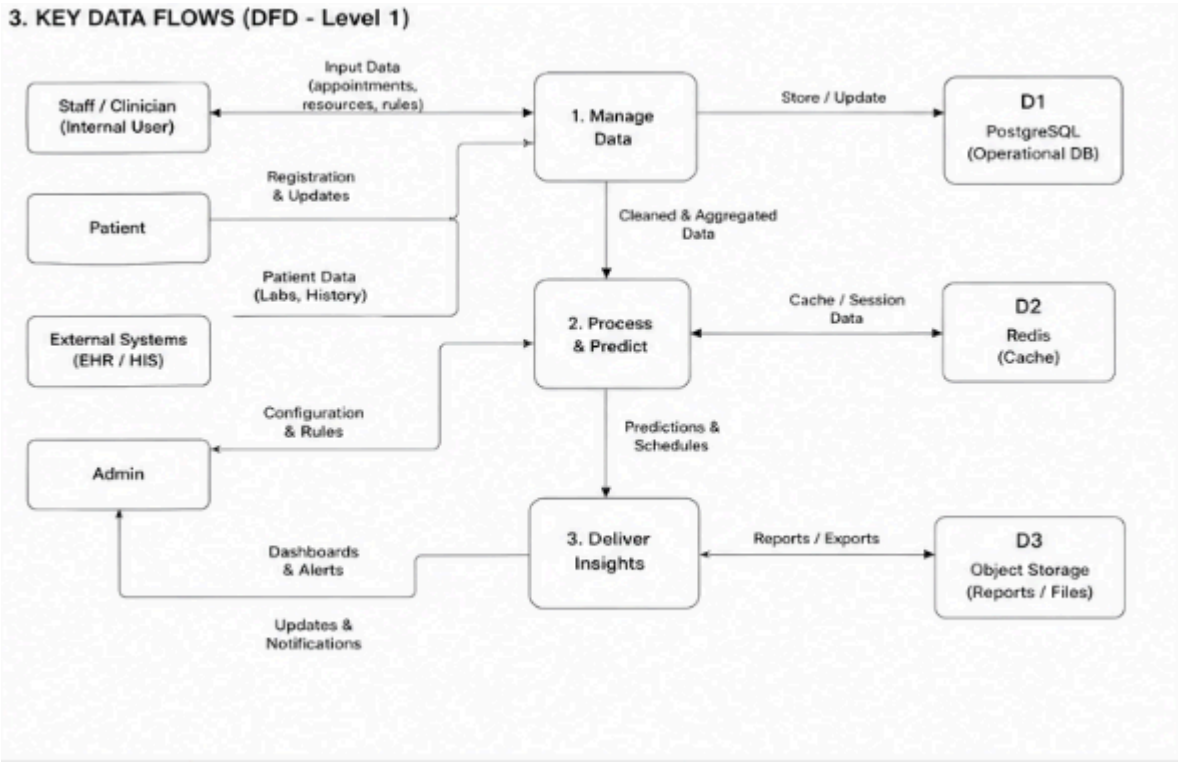
2.3.1. Data Flow Diagram (DFD) — Level 1

External Entities: Patient, Nurse, Clinician, Insurance API, Anthropic Groq and OpenAI API

Process	Inputs	Outputs	Data Stores Accessed

P1: Patient Intake	Symptom form data (name, IC, symptoms, vitals, language)	Raw PatientRecord (pending)	DS1: patients table
P2: LLM Triage Inference	Raw PatientRecord + system prompt template	Triage result JSON (level, reason, department)	DS2: triage_results table; DS3: Redis cache
P3: Queue Management	Confirmed triage result	Updated queue; nurse dashboard push notification	DS1: patients; DS2: triage_results
P4: Routing Engine	Confirmed triage level + department availability	Bed/doctor assignment	DS4: ward_availability table
P5: Insurance Pre-Auth	Patient insurance ID + proposed ICD-10 code	Pre-auth request to external Insurance API	DS5: insurance_records table
P6: Clinician Copilot	Patient visit history + current intake + copilot prompt	Structured clinical summary (AI-generated)	DS1: patients; DS6: visit_history table

DFD Law Compliance — Black Hole Check: Every process has at least one input and one output. No data store is written without a subsequent read. External entities (Patient, Nurse, Clinician) are the only sources/sinks. The Anthropic Groq and OpenAI API is modelled as an external service entity receiving structured prompts and returning structured responses.



2.3.2. Normalised Database Schema (3NF ERD)

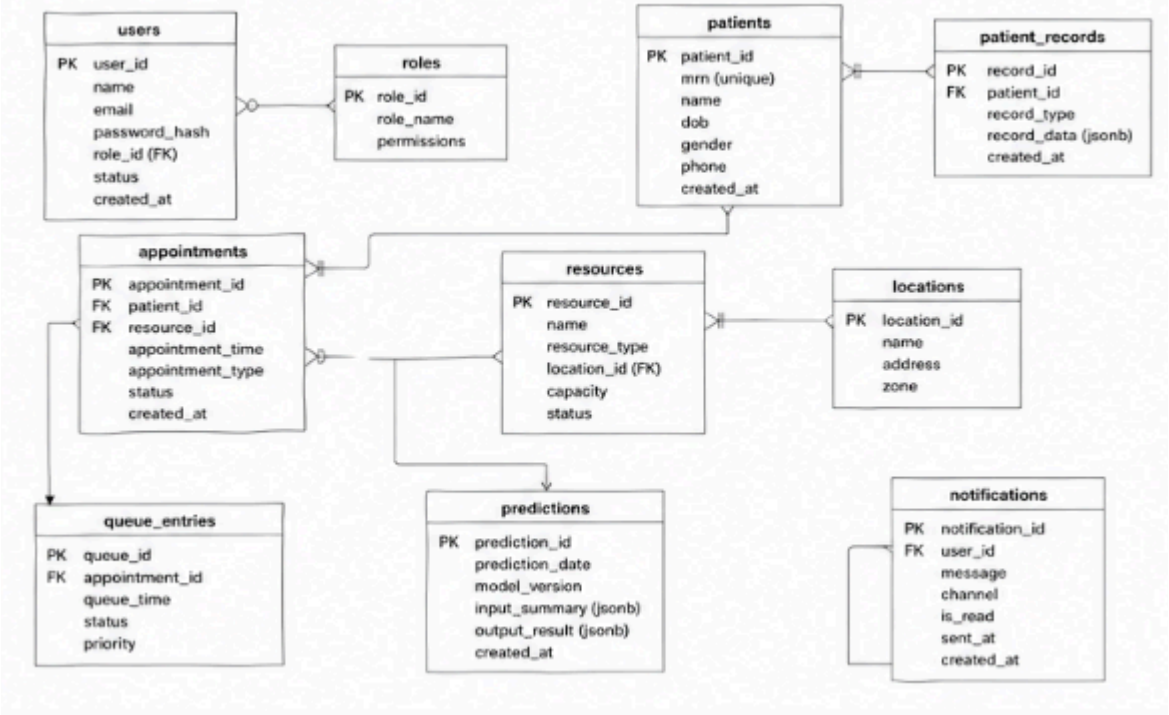
All tables are designed to Third Normal Form (3NF). Junction tables resolve M:N relationships. No transitive dependencies exist.

Table	Primary Key	Foreign Keys	Key Columns	Purpose
patients	patient_id (UUID)	—	ic_number, full_name, dob, gender, language_pref, insurance_id	Core patient registry; 1NF-3NF compliant — no repeating groups
triage_results	triage_id (UUID)	patient_id → patients	triage_level (P1–P5), urgency_reason, recommended_dept, confidence_score, ai_raw_response, confirmed_by, confirmed_at	Stores LLM triage output + nurse confirmation

intake_sessions	session_id (UUID)	patient_id → patients	symptoms_text, vitals_json, language_detected, submitted_at	Stores each intake submission; separates intake from triage to avoid redundancy
ward_availability	ward_id (UUID)	—	department_name, available_beds, attending_doctor_id, last_updated	Real-time ward occupancy; updated by routing engine
visit_history	visit_id (UUID)	patient_id → patients, triage_id → triage_results	diagnosis_icd10, treatment_notes, discharge_at, attending_doctor_id	Longitudinal patient history for copilot context
insurance_records	record_id (UUID)	patient_id → patients	insurer_name, policy_number, preauth_status, preauth_submitted_at	Tracks insurance pre-auth lifecycle
users (staff)	user_id (UUID)	—	full_name, role (nurse/clinician/admin), department, hashed_password	Authentication table for clinical staff; roles enforce RBAC

M:N Relationships resolved: visit_history acts as a junction between patients and triage_results (a patient may have many visits; each visit links to one triage result). vitals_json uses JSONB in PostgreSQL to store flexible vital sign readings without breaking 3NF for structured fields.

4. NORMALIZED DATABASE SCHEMA (ERD - 3NF)



3. Functional Requirements & Scope

3.1. Minimum Viable Product (MVP)

The following 5 core features will be built and demonstrated at UMHackathon 2026:

#	Feature	Description
1	Multilingual Patient Intake Portal	Patients submit symptoms and basic vitals via a web form in Bahasa Malaysia or English. Language is auto-detected; the form adapts accordingly. Data is validated and submitted to the backend for triage processing.
2	AI-Powered Triage Engine (LLM)	Submitted intake data is sent to OpenAI API via a structured prompt. OpenAI returns a triage level (P1–P5), urgency reasoning, and recommended department in JSON format. The result is stored in the database and surfaced to nursing staff in real time.
3	Nurse Triage Dashboard	A real-time queue dashboard showing all triaged patients, their AI-assigned triage level, and a one-click confirm/override control. Nurses can escalate or downgrade the AI recommendation with a mandatory reason field.
4	Smart Routing Engine	After triage confirmation, the system queries live ward availability and auto-assigns the patient to the most appropriate bed and attending physician. Routing logic prioritises P1/P2 patients and balances load across available doctors.
5	AI Clinician Copilot	When a clinician opens a patient record, the system generates a structured clinical summary using Gemini API — summarising symptoms, visit history, medications, allergies, and the AI triage recommendation with source references.

3.2. Non-Functional Requirements (NFRs)

Quality	Requirement	Implementation
Scalability	System must handle up to 500 concurrent patient intake submissions during peak hours (8–10 AM hospital peak)	Stateless backend deployed on Railway with horizontal scaling; Neon PostgreSQL auto-scales connection pool; Redis caches repeated triage queries
Reliability	Triage results must be persisted with 99.5% uptime; a failed LLM call must not leave orphaned patient records without a triage result	Fallback triage (P3 default) triggered on LLM timeout or parse failure; DB transactions ensure atomicity of intake + triage INSERT
Maintainability	LLM prompt templates must be version-controlled and editable without redeployment; each core feature is independently testable	Prompts stored as versioned .txt files in /prompts directory; modular Express routers per feature (intake, triage, routing, copilot)
Token Latency	OpenAI and Groq API triage call must return a parsed result within 5 seconds under normal load (p95)	Streaming enabled (SSE) so first token appears within 1–2s; hard timeout of 8s with fallback; loading state shown on nurse dashboard
Cost Efficiency	Average LLM token cost per patient session must not exceed USD $0.02	Intake text truncated to 500 tokens max; patient history compressed to 1,500 tokens before injection; Redis caches triage results for identical symptom patterns (TTL 30 min); OpenAI model used for insurance pre-auth (lower cost)
Security & Privacy	Patient IC numbers and medical data must be encrypted at rest and in	All API traffic over HTTPS/TLS 1.3; PostgreSQL encryption at rest (Neon); IC numbers hashed in application layer; JWT with 1-hour expiry for staff sessions

	transit; PDPA (Malaysia) compliance required	

3.3. Out of Scope / Future Enhancements

The following were planned for the CareFlow 'Grand Vision' but are excluded from UMHackathon 2026:

- In-app real-time chat between patient and assigned nurse during waiting period
- Integration with Malaysia's MySejahtera / MyHEALTH patient records system for automatic history retrieval
- Predictive bed occupancy forecasting using historical admission data (ML model, separate pipeline)
- Native iOS & Android mobile application for patient intake (current scope is mobile-responsive web)
- Full insurance claims submission and payment gateway integration (pre-auth only is in scope)
- Voice-based symptom input with Whisper ASR for elderly patients
- Multi-hospital deployment with cross-facility patient routing

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation

4.1.1. Grayscale Rollout & A/B Testing

Post-hackathon deployment strategy: The AI triage feature will be introduced to 10% of intake sessions initially, with nurses manually triaging the remaining 90% as control. System metrics (triage accuracy, nurse override rate, time-to-triage) are monitored for 72 hours. If nurse override rate remains below 15%, rollout expands to 50%, then 100%. A/B testing will compare two prompt variants — one using structured vitals-first ordering vs. symptoms-first — to determine which produces lower override rates and higher nurse confidence scores.

4.1.2. Emergency Rollback (ER) & Golden Release

If the AI triage override rate exceeds 30% in any rolling 1-hour window, or if Groq and OpenAI API error rate exceeds 5%, an Emergency Rollback is triggered. The system automatically reverts to the 'Golden Release' — the last validated stable build — via the Railway deployment rollback mechanism. During ER, all triage falls back to manual nurse assessment. A status banner is displayed on the nurse dashboard: 'AI Triage Temporarily Unavailable — Manual Assessment Mode.'

4.1.3. Priority Matrix

Priority	Trigger Condition	Action
P1 — Critical	Groq and OpenAI API error rate > 5% OR triage fallback rate > 10% in 30 min	Immediate ER triggered; Engineering lead paged; fallback to manual triage; Incident report filed
P2 — High	Nurse override rate > 20% in 1-hour window	Prompt engineering review; A/B test new prompt variant; flag for clinical review of AI outputs
P3 — Medium	Average triage latency > 5s (p95)	Investigate LLM service bottleneck; check Redis cache hit rate; consider prompt shortening
P4 — Low	Database query latency > 200ms on queue fetch	Add composite index on (patient_id, triage_level, submitted_at); review ORM query plan

4.2. Monitoring

The following monitoring plan is implemented for system health tracking:

Metric	Tool	Threshold	Alert Action
Groq and OpenAI API response time (p95)	Custom middleware logging + Datadog / BetterStack	> 5 seconds	Slack alert to #careflow-ops channel; auto-retry with exponential backoff
Groq and OpenAI API error rate	Error tracking middleware (Express)	> 5% in 30-min window	Trigger P1 ER; switch to manual triage mode
Triage nurse override rate	Custom analytics table (overrides_log)	> 20% in 1-hour window	Slack alert; prompt review scheduled
Database connection pool	Neon dashboard + application health endpoint	> 80% pool utilisation	Scale connection pool; alert DevOps
Backend API uptime	UptimeRobot / BetterStack	< 99% in 24h	PagerDuty alert; ER assessment
Token usage per session	Anthropic Usage API polling (every 15 min)	> $0.02/session average	Alert cost owner; enable stricter truncation

4.3. Assumptions

The following operational and environmental conditions are assumed valid during development and deployment:

- Hospital staff (nurses, clinicians) have access to a modern browser (Chrome/Edge) on ward workstations or tablets.
- Patients self-registering have sufficient mobile data or Wi-Fi access to complete the intake form.
- Anthropic Groq and OpenAI API maintains its published 99.9% uptime SLA for the duration of the hackathon and pilot period.

- Patient IC numbers provided during intake are accurate; no identity verification integration is in scope.
- ICD-10 code mapping for insurance pre-auth is based on Groq and OpenAI 's inference from symptom data; a pharmacist or clinician is responsible for final code confirmation.
- The hospital's existing ward management system does not need to be integrated; ward availability is managed via CareFlow's own ward_availability table, updated manually by admin staff initially.

4.4. External Dependencies

Tool / Service	Purpose	Risk Level	Mitigation Strategy
OpenAI and Groq API ()	Core LLM reasoning engine — handles triage inference, copilot summarisation, and insurance pre-auth prompt generation	HIGH — Core product fails without this API	Graceful degradation: fallback to P3 default triage + manual mode; retry with exponential backoff (max 3 retries); cached results served from Redis during outage
Neon PostgreSQL	Primary persistent data store for all patient records, triage results, and session data	MEDIUM — Data at risk if instance goes down	Neon automatic failover to read replica; daily automated backups; connection pooling via PgBouncer
Upstash Redis	Caching layer for frequent triage patterns; session token storage	LOW — Fallback is direct DB query	If Redis unavailable, all requests bypass cache and hit DB directly; no data loss risk
langdetect (npm)	Auto-detection of patient input language (BM vs EN) for prompt routing	LOW — Library is lightweight and offline	Fallback: default to English prompt if detection confidence < 0.8

5. Project Management & Team Contributions

5.1. Project Timeline (10-Day Sprint)

Phase	Days	Tasks
Planning & Architecture	Day 1–2	Requirements gathering; SAD drafting; system architecture design; database schema design (3NF); LLM prompt schema definition; GitHub repo setup and branch strategy
Core Backend + DB	Day 3–4	Prisma schema setup and migrations; REST API scaffolding (Express); Patient intake API (/api/intake); LLM Service module with Groq and OpenAI API integration; triage result persistence
AI Triage Engine	Day 4–5	Prompt engineering for triage (P1–P5 classification); bilingual (BM/EN) prompt testing; JSON schema validation for Groq and OpenAI responses; fallback/degraded mode implementation; Redis caching integration
Frontend — Patient Intake	Day 4–6	Next.js project scaffold; multilingual intake form (BM/EN toggle); form validation; API integration; mobile-responsive layout (Tailwind)
Nurse Dashboard + Routing	Day 6–7	Real-time triage queue UI; confirm/override controls; routing engine logic; ward availability table; smart assignment algorithm
Clinician Copilot + Insurance Pre-Auth	Day 7–8	Copilot prompt engineering; clinical summary UI; Insurance pre-auth prompt + external API stub; patient history compression logic
Integration Testing & Bug Fixes	Day 8–9	End-to-end flow testing; edge case handling (LLM timeout, empty fields, language edge cases); performance testing (simulated load); PDPA data handling review

Demo Prep & Documentation	Day 9–10	Demo script preparation; SAD finalisation; slide deck; video demo recording; GitHub README polish; final deployment to Vercel + Railway

5.2. Team Member Roles

Name	Role	Key Responsibilities
Dennis Heng Shu Yi	Full-Stack Lead & LLM Integration	System architecture; Groq and OpenAI API integration; LLM Service module (prompt engineering, JSON parsing, fallback); backend REST API (Express); Prisma schema; GitHub management
Ng Sean Sean	Backend Engineer & Database Design	PostgreSQL schema design (3NF); Prisma migrations; routing engine logic; ward availability API; Redis caching setup; backend API endpoints (routing, insurance)
Lei Wing Teng	Fullstack Engineer — Patient Portal & Claims System	Next.js Patient Intake Portal; multilingual form (BM/EN); Insurance Form Generation; API integration; mobile-responsive design (Tailwind / shadcn)
Chu Wei Lyn	Frontend Engineer — Nurse & Clinician Dashboards	Nurse triage queue dashboard; confirm/override UI controls; Clinician Copilot UI; real-time queue polling / WebSocket integration; data visualisation
Denise Koh Wei Sin	QA & Product	Test case design (intake → triage → routing flow); edge case documentation; nurse override rate tracking; insurance pre-auth stub testing; demo script; presentation deck

5.3. Recommendations

Based on architectural decisions made during development, the following recommendations are proposed for post-hackathon scalability, performance, and reliability:

- Redis Prompt Caching: Cache identical or near-identical symptom patterns (fuzzy hash) with a 30-minute TTL to reduce Groq and OpenAI API call volume by an estimated 20–35% during peak hours.
- Prompt Versioning System: Implement a versioned prompt registry (v1.0, v1.1, etc.) with A/B testing support so prompt improvements can be validated against live nurse override rates before full rollout.
- WebSocket for Real-Time Queue: Replace polling with WebSocket (Socket.io) on the nurse dashboard to reduce latency from ~2s poll interval to sub-100ms real-time push — critical for P1/P2 patient escalations.
- LLM Cost Tiering: Route insurance pre-auth calls to Groq and OpenAI -haiku (lower cost, faster) while reserving Groq and OpenAI -sonnet for triage and copilot calls where reasoning quality is critical.
- PDPA Compliance Audit: Before any hospital pilot, conduct a formal Personal Data Protection Act (Malaysia) audit — ensure patient IC numbers are pseudonymised in logs, LLM prompts do not transmit raw IC numbers, and data retention policies are implemented (auto-delete after 7 years per PDPA).
- Offline Fallback Mode: For hospitals with unreliable internet, implement a local cached triage decision tree (rule-based) as a secondary fallback beyond the current P3 default — ensuring clinically safe degradation even without any external API access.